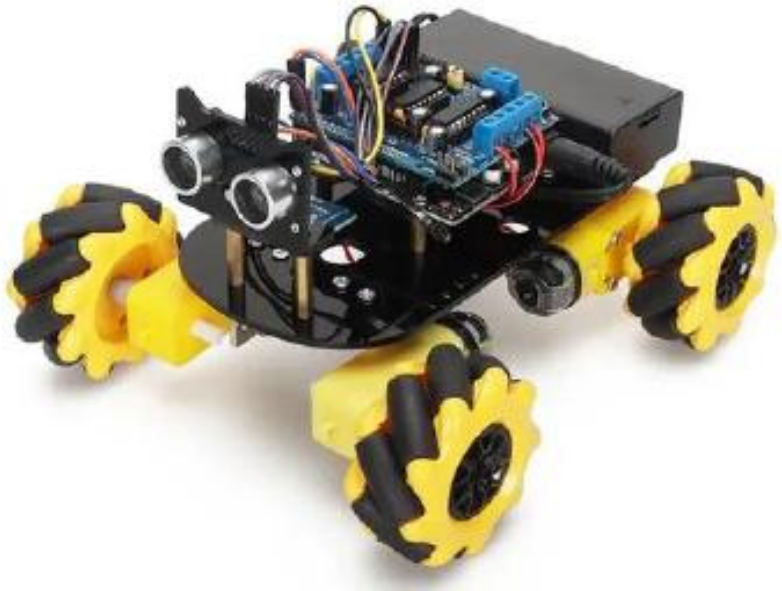


Projet ROVER

Sprint 3 - Navigation

Robotique - Python



Gp221
2025-2026

Sommaire

- I. Sprint 3 dans le projet Rover global
- II. Objectifs du sprint 3 – changement de d’approche (mode projet)
- III. Déroulement des séances
- IV. Rendus / Notation
- V. Rappels techniques
 - 1. Modules Python
 - 2. Structure du rover et protocoles de communication
 - 3. Communication avec le Pico / Rover
 - 4. Point rapide sur l’asservissement
- VI. Analyse du code de contrôle de départ
- VII. Choix du projet

Projet ROVER – vue globale

Sprint 1

Concevoir un Rover **démontable** et assembler tous les composants sans problèmes de câblage ni de frottements.

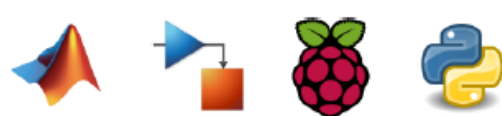
Thèmes : Dimensionnement mécanique et électrique, CAO, Impression 3D, Découpe laser



Sprint 2

Commander les déplacements du Rover avec un microcontrôleur et implémenter un correcteur de trajectoire.

Thèmes : Asservissement de vitesse, Automatique, Python, Modélisation et simulation avec Matlab et Simulink



Sprint 3

Naviguer de façon autonome et détecter des obstacles pour adapter la trajectoire du Rover.

Thèmes : Détection ultrason, Traitement des données



Sprint 4

Piloter le Rover à distance, créer une interface de contrôle et supervision.

Thèmes : Contrôle à distance via Wi-Fi, API Python



Objectifs du sprint 3

Changement de méthode

- Ecrire les commandes de **haut niveau** pour rendre le rover **autonome**
 - Bas niveau : travail sur les protocoles de communication ou l'électronique (sprint 2)
 - Haut niveau : fonctions simples pour des mouvements élémentaires (avancer, tourner,...)
 - Autonome : pas de pilotage en temps réel du rover (il fait sa mission seul)
- Travailler sur un petit projet au choix en équipe en utilisant les commandes
 - Exemple : éviter un obstacle, sortir d'un labyrinthe, comparaison théorie/réalité,...
 - Faisabilité à vérifier
- Passage en **mode projet**:
 - Sprint 2: mode TP (liste d'instructions à effectuer)
 - Sprint 3 : mode projet
 - Répartition du travail entre membres de l'équipe
 - Définition de sous objectifs
 - Travail en semi-autonomie (je suis là pour aider !)
 - Soutenance à la fin

Objectifs en résumé

I. Ecrire les fonctions :

- Avancer (/ reculer) de $x \text{ cm}$
- Tourner de α degrés
- Déplacement latéral de $x \text{ cm}$
- Mesurer la distance de l'obstacle à $x \text{ cm}$

II. S'amuser avec ces fonctions pour faire faire une mission au rover.

Déroulement des séances

Module de 20h : 5 x 4h

Séances n°	Objet du de la séance
1 (4h)	• Présentation des objectifs et du planning des séances • Mini-cours introductif aux simulations numériques • Fonctions de contrôle des mouvements fonctionnels (avancer, tourner, dpl latéral)
2 (4h)	• Sonar + Servo fonctionnels (vérifications) • Réflexion sur le mini-projet d'applications • Début du travail sur le mini-projet
3 (4h)	• Travail sur le mini-projet
4 (4h)	• Fin du développement • Speech sur les bonnes pratiques de présentation
5 (4h)	• Finalisation • Présentation du projet et démonstration

Rendus / Notes

- Présentation orale en groupe à la dernière séance:
 - Durée : 10 min / groupe + 5/10 min de questions/commentaires
 - Pas de lecture/présentation de code **MAIS** il faut rentrer dans le détail technique
 - Une démonstration en direct ou en vidéo serait un plus
- Fin du projet : les slides et le code (+ fichiers) doivent être rendus sur Moodle
- La motivation et les initiatives comptent dans la note !

Sprint 2 → Sprint 3

- TP du Sprint 2 :
 - Utilisation de Matlab/Simulink pour approcher simplement l'asservissement (PID)
 - Manipulation du module Python qui contrôle les moteurs et qui récupère le nombre de tours que font les roues pour s'assurer que la roue tourne à la bonne vitesse (si elle ne tourne pas à la bonne vitesse on applique une correction → asservissement)
- Sprint 3 :
 - On oublie l'aspect asservissement (PID etc...)
 - On ne fait que du Python (pas de Matlab ou autre)
 - On part tous avec le même **module** de contrôle corrigé (sur Moodle)
 - On écrit un « main.py » qui importe le **module** « IPSA_Rover_lib.py » fourni
 - On n'utilise qu'une seule fonction « control_motor_speed » de ce script

Rappels techniques

Rappels Python : Modules

- Bibliothèques, libraries, modules : globalement identique
- Comment lire un fichier qui n'est pas un fichier Python avec Python ?
 - OPEN
- Comment lire/exécuter un fichier Python avec Python ?
 - IMPORT
- Pourquoi certains scripts Python finissent par une partie :

```
if __name__ == "__main__":
```

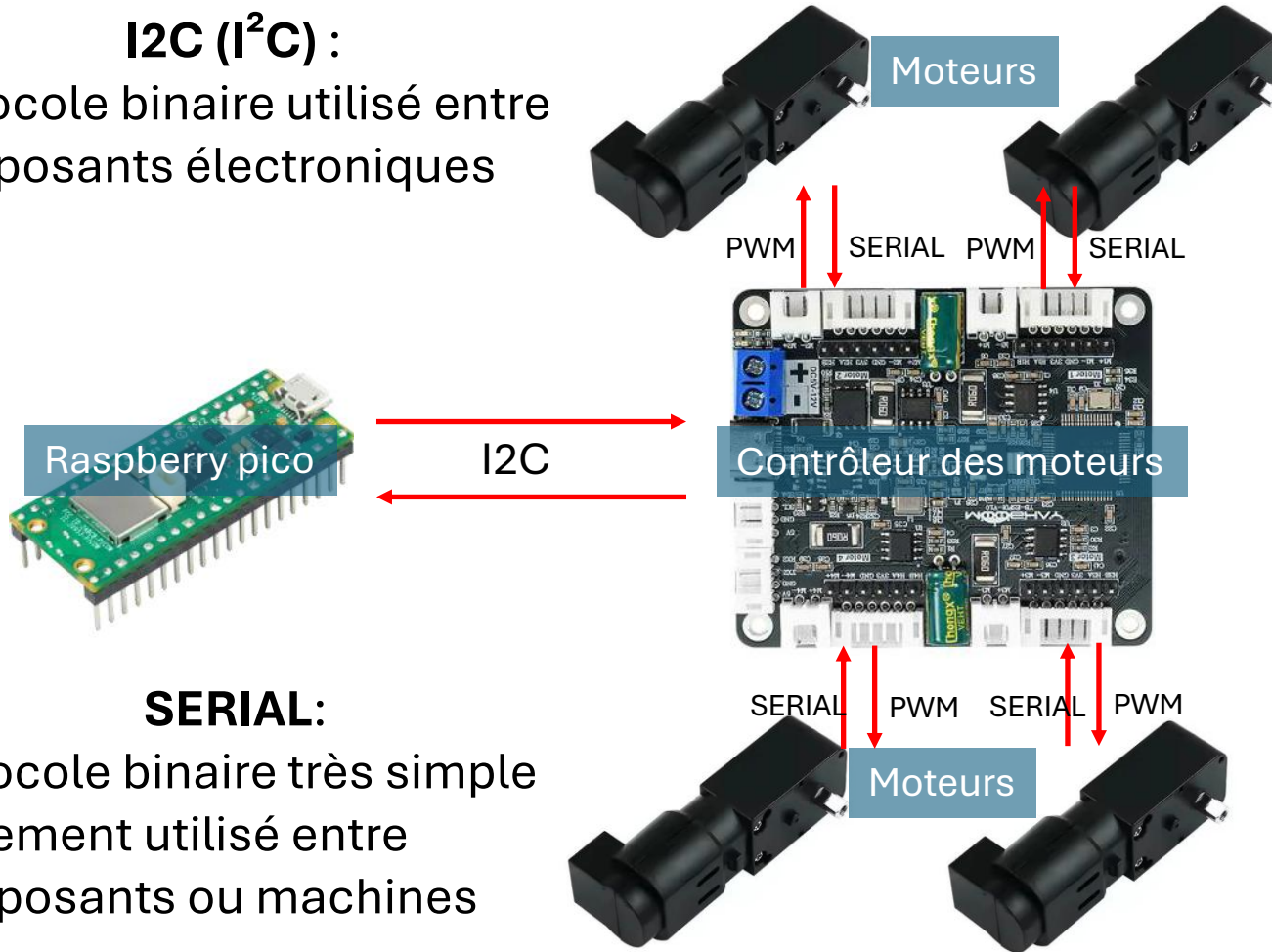
→ pour avoir une zone de code qui s'exécute quand c'est ce script qui est **exécuté** mais qui ne s'exécute pas si le script est **importé** !

Structure du Rover et protocoles

Partie motorisation

I2C (I²C) :

Protocole binaire utilisé entre composants électroniques



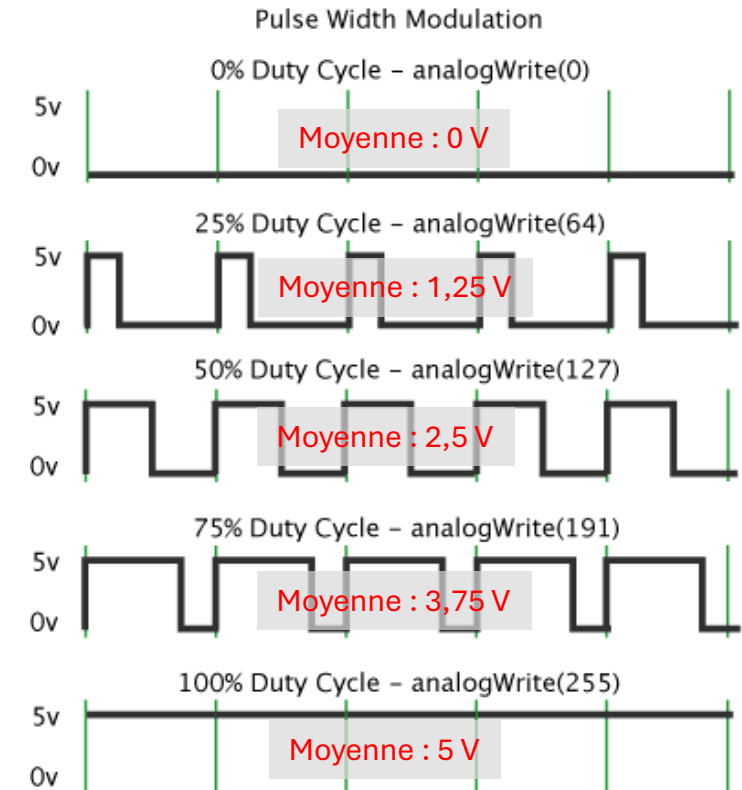
SERIAL:

Protocole binaire très simple également utilisé entre composants ou machines

Note : dans ce *sprint* vous ne manipulerez pas directement ces protocoles !

PWM (Pulse Width Modulation) :

Produire un signal analogique (valeurs continues de tension) avec un signal numérique (que des 0V ou 5V)



Duty Cycle (EN) : Rapport Cyclique (FR)

Les ticks de encodeur

la tique de l'encoQuoi ?

- Encodeur : c'est le capteur qui est inclus dans chaque moteur est qui sert à compter le nombre de tour qu'à fait le moteur.
- Un tick est une impulsion envoyée par le contrôleur des moteurs au Raspberry Pico
- Mais le moteur à un réducteur (pour avoir du couple)
 - Donc un tick ne correspond pas à un tour
 - Il y a :

$$N \approx 2340 \text{ ticks/tours}$$

Note : dans ce *sprint*, il n'y a pas besoin de lire les ticks pour asservir les moteurs (c'est déjà fait) mais cela sera utile pour vérifier la distance parcourue par le rover.

Commander un Servo-moteur



- Le servo-moteur est un moteur qui maintient un angle précis en fonction du signal qu'il reçoit.
 - $0^\circ \leftrightarrow 180^\circ$
- Le signal est du PWM à 50 Hz (→ revient à 5V toutes les 20 ms)
- Contrôle des angles :
 - **0°**: 5% du rapport cyclique (→ $1000 \mu s$ (1 ms))
 - **180°**: 10% du rapport cyclique (→ $2000 \mu s$ (2ms))

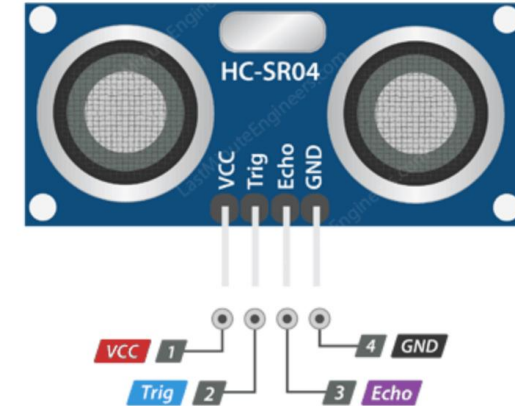
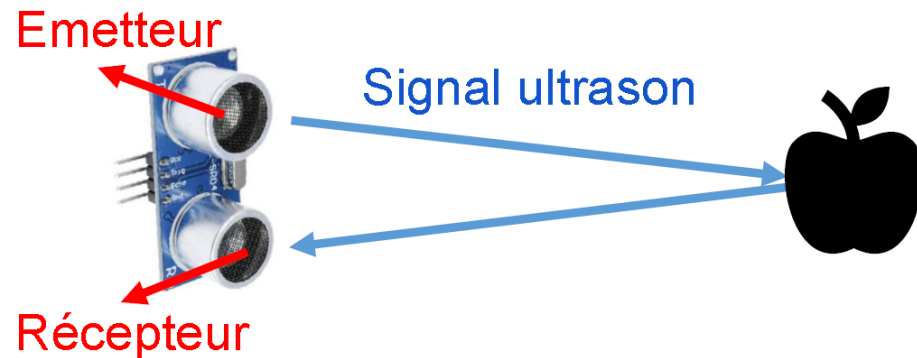
Exemple:

Si on veut envoyer le moteur à 90° d'angle on envoi un créneau (la commande) de $1500 \mu s$

Note : La plupart des micro-contrôleurs (comme le Pico) peuvent facilement générer du signal PWM

Mesurer les distances avec un Sonar

Capteur à ultrasons :



Vitesse du signal = vitesse du son $c = 340 \text{ m/s}$

d est la distance entre le capteur et l'obstacle

Δt est le temps que le signal met pour parcourir la distance $2.d$ (un aller-retour)

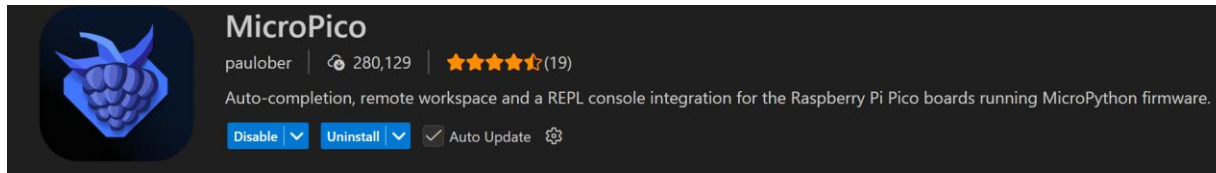
$$c = \frac{2d}{\Delta t} \Rightarrow d = \frac{c \cdot \Delta t}{2}$$

1. Vcc : 5 V
2. Trig (Trigger) (Pin GP21) : signal de déclenchement de l'onde incidente (envoyé par le pico)
3. Echo (Pin GP20): signal déclenché par le retour de l'écho (lue par le pico)
4. GND : Masse

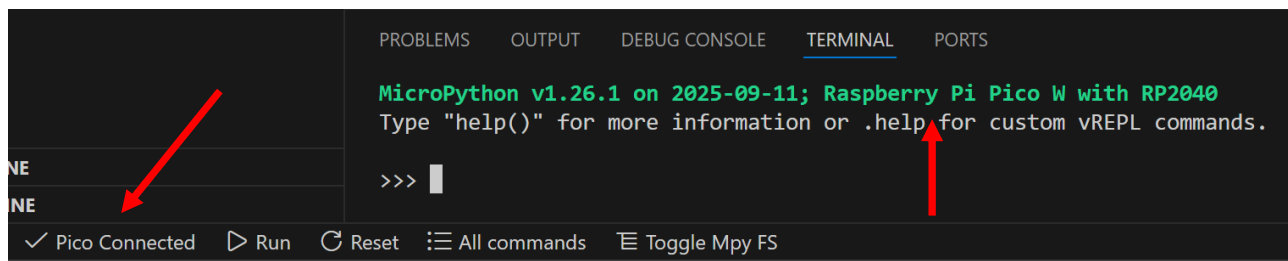
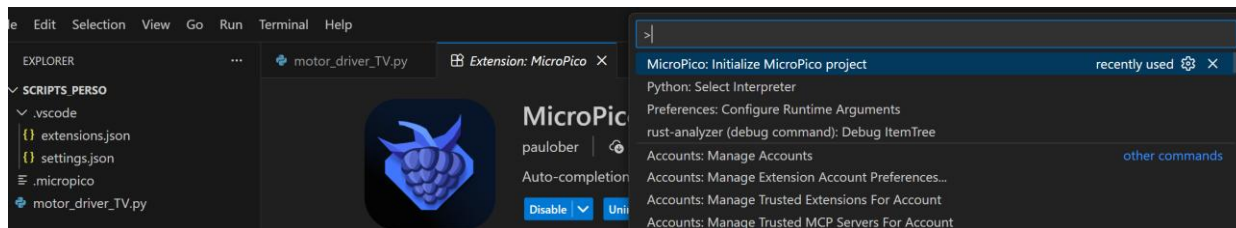
Interface IDE – VSCode ou Thonny

Visual Studio Code (Microsoft)

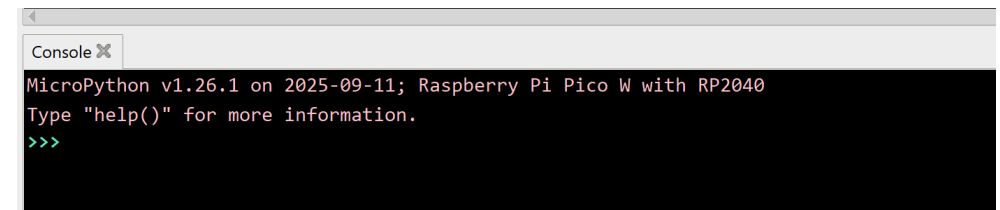
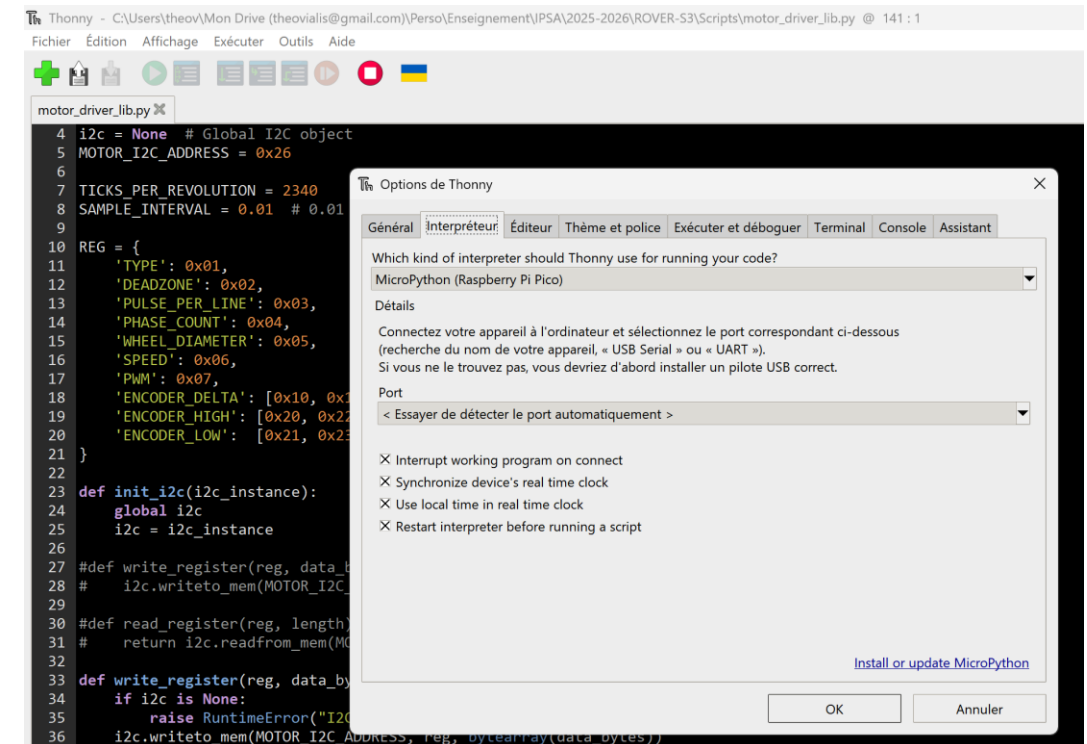
- Installer l'extension : MicroPico (paulober)



- Initialiser un projet Pico en faisant :
 - Ctrl + MAJ + P ➔ Initialize MicroPico projet



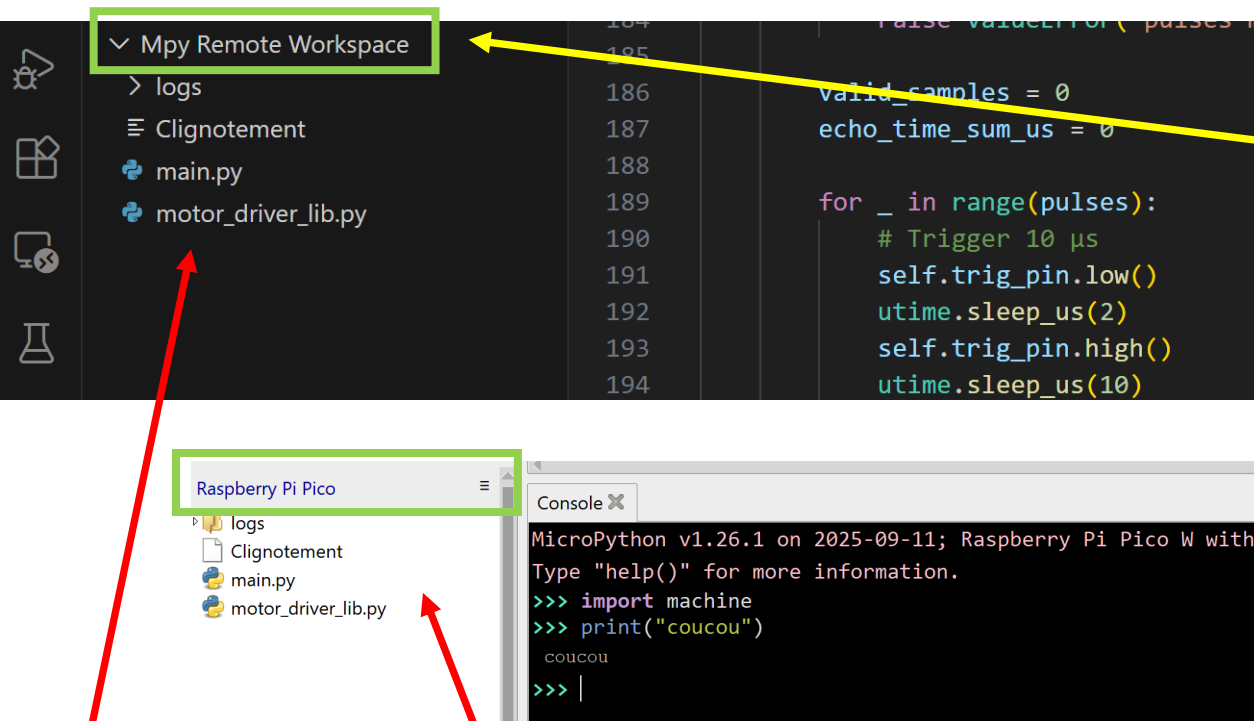
Thonny



Execution du code sur le Pico

Exécution automatique

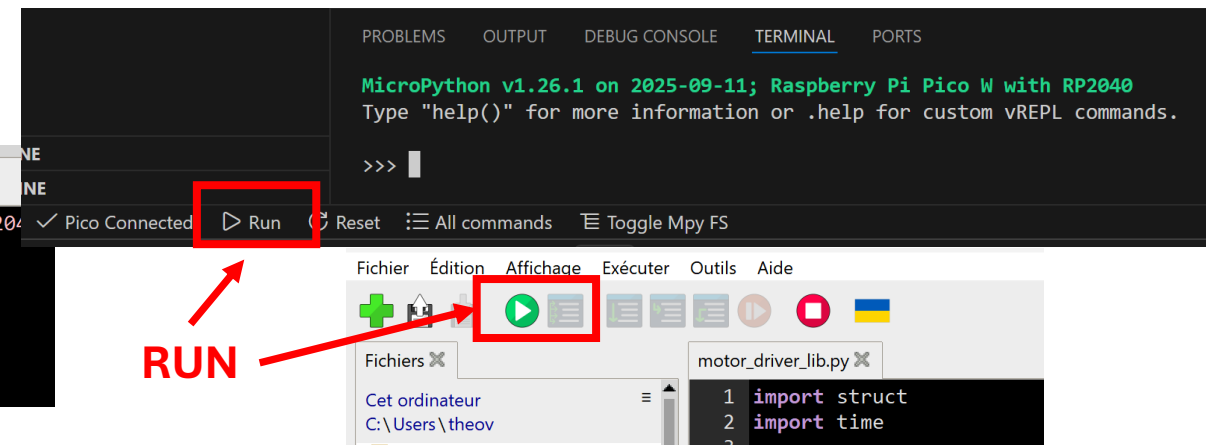
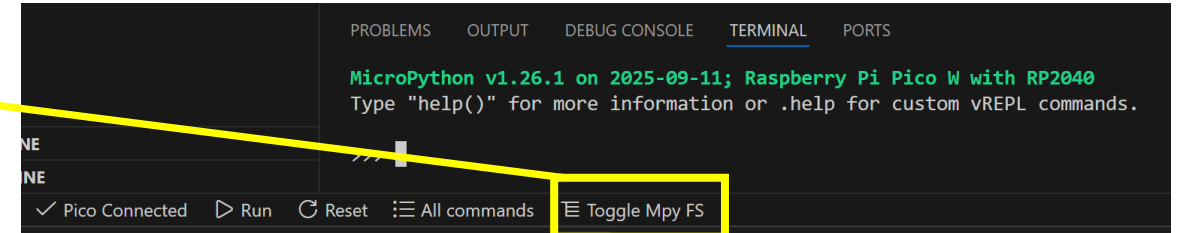
- Un script appelé « main.py » doit être enregistré dans la mémoire interne du Pico (voir les images)



- Les bibliothèques doivent être dans la mémoire du Pico (et pas sur le PC)

Exécution en direct via REPL

- REPL (Read-Eval-Print-Loop) : permet d'exécuter le python du Pico directement sur le PC (le calcul est fait sur le Pico pas sur l'ordinateur)



Automatisme & Asservissement

- Faire **fonctionner un engin automatiquement**:

→ il faut lui ajouter une forme d'**intelligence** (un algorithme de décision ou de correction).

- On distingue alors deux grands modes de fonctionnement :

Boucle ouverte

- Le système exécute une commande **prévue à l'avance** (ex. allumer un moteur pendant 2 s, sans vérifier la position)
- Simple, mais sans adaptation à la réalité.

Boucle fermée (asservissement)

- Le système **mesure un état, compare** à une consigne et **corrige** automatiquement.

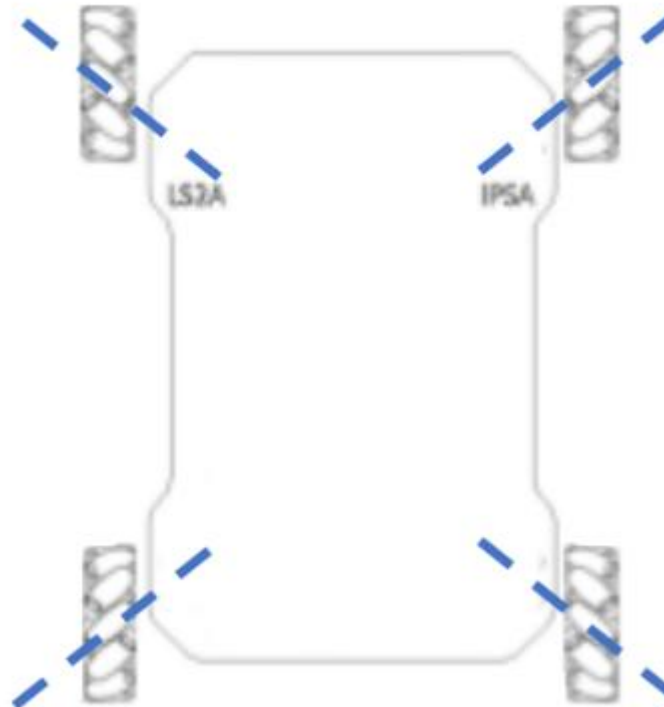
$$e(\text{erreur}) = \text{consigne} - \text{mesure}$$

- Exemple : stabiliser une altitude, garder une orientation, viser une position.

Type	Expression	Effet principal	Complexité
Bang-Bang	commande max/min selon le signe de l'erreur	Réactif mais peu précis	Très simple
P (proportionnel)	$u = K_p e$	Ramène vers la consigne, mais peut osciller	Simple
PD (prop. + dérivé)	$u = K_p e + K_d \dot{e}$	Amortit les oscillations	Moyen
PID (prop. + dérivé + intégral)	$u = K_p e + K_i \int e \, dt + K_d \dot{e}$	Précis et stable	Avancé

Pour bien commencer...

Attention à l'orientation des roues



Point de départ du sprint 3

1. Motorisation

- Créer un nouveau fichier (qui doit s'appeler « main.py » si exécution automatique)

- Importation des modules :

```
1 import time
2 from IPSA_Rover_Lib import IpsaRoverLib
3
4 driver = IpsaRoverLib()
```

- Création de l'objet de base :

Méthode (fonction) de déplacement:

control_motor_speed(vit_m1, vit_m2, vit_m3, vit_m4)

- vit_m1 : vitesse moteur 1 (valeurs entre -1000 et +1000)
- Valeurs négatives pour avancer (conception du rover)

```
6 driver.control_motor_speed(-200, -200, -200, -200)
7 time.sleep(3)
8 driver.control_motor_speed(0, 0, 0, 0)
```

Exemple : avance à 200 (20% vit max) pendant 3 sec et arrêt

Attention : fonction non bloquante !

Vérification de la rotation :

[vit_1, vit_2, vit_3,]= read_encoder_deltas()

- Donne le nombre de ticks pour 10 ms

```
ticks = driver.read_encoder_deltas() #compte combien de ticks en 10 ms
print(f"Nombre de ticks : {ticks}") #pour chaque moteur
RPM1 = driver.calculate_rpm(ticks[0]) # calcul du rpm (rotation per minute) pour moteur 1
print(f"RPM : {RPM1}")
```

RPM = calculate_rpm(nbr_ticks)

- Rpm : tours / minute
- Fait simplement le calcul (avec 2340 ticks/tour et 10 ms)

Point de départ du sprint 3

2. Servo et Sonar

Rotation du servo-moteur :

`set_servo_pulse_us(pulse_us)`

- Ordonne au Servo d'avoir l'angle correspondant à la valeur du pulse :
 - 1000 μ s = 0°
 - 2000 μ s = 180° (environ)

Exemple : fait tourner le servo entre 0° et 180° en 20 pas espacés de 100 ms :

```
for us in range(1000,2000,50):  
    driver.set_servo_pulse_us(us)  
    time.sleep(.1)
```

Mesure de la distance devant le sonar:

`temps_ms= read_sonar_echo_time_ms()`

- Mesure le temps d'aller retour de l'onde sonore donnée en millisecondes

Exemple : demande 10x le temps d'aller-retour de l'onde ultrasonique au Sonar

```
for mes in range(10):  
    echo_time_ms = driver.read_sonar_echo_time_ms()  
    print(f"Temps de vol de l'onde : {echo_time_ms}")  
    time.sleep(.5)
```

Maintenant (1^{ère} séance)

Dans un fichier main.py

```
def linear_move_cm(distance_cm):  
    """  
    votre code / fonction qui fait avancer le rover de "distance_cm" cm  
    """  
  
def turn_degree(degree):  
    """  
    votre code / fonction qui fait tourner le rover de "degree" degrés sur lui-même  
    """  
  
def lateral_move_cm(distance_cm):  
    """  
    votre code / fonction qui fait déplacer latéralement le rover de "distance_cm" cm  
    """
```

Aide : vous allez avoir besoin de savoir de combien de cm le rover bouge pour un tour de roue...